Повне повторення: Arduino і мова C

Поглиблення до уроку 1 — 2МАК, SST-INF-01c — Увесь firmware минулого року

15/10/2026

Примітка

Цей матеріал збирає **увесь** **firmware** минулого року, з більшою кількістю деталей і прикладів, ніж матеріал A01 (який залишається підсумком уроку 15/10). Він потрібен тим, хто має **повторити**, і тим, хто **прийшов** **цього** **року**. Це основа **контрольної** **з** **повторення** **29** **жовтня**. Джерела: матеріал про Arduino, який використовували минулого року, матеріали з C і з firmware, і перша година уроку 15/10.

1 Плата Arduino Uno

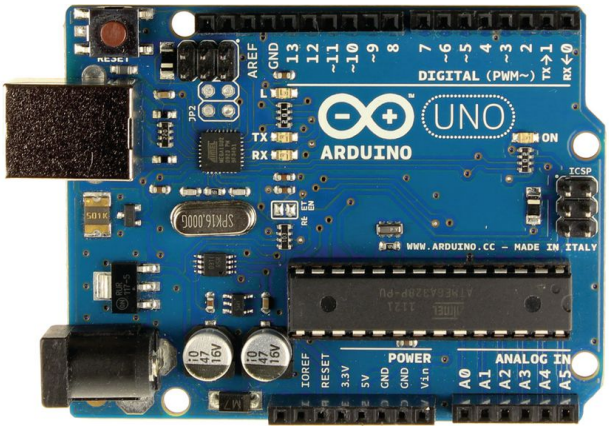


Рис. 1: Плата Arduino Uno: ліворуч роз'єм USB і гніздо живлення, угорі цифрові виводи 0–13, унизу виводи живлення (5V, GND) і аналогові входи A0–A5.

- **Роз'єм** **USB**: має подвійне призначення. Він **живить** плату від 5 В порту USB комп'ютера і дозволяє **обмінюватися** **даними** з Arduino (дуже корисно, щоб зрозуміти, що відбувається всередині, через монітор послідовного порту).
- **Гніздо** **живлення**: у кутку є маленьке чорне гніздо, у яке вставляють штекер (*jack*) з внутрішнім отвором 2,1 мм. Напругу, що надходить, **регулятор** **напруги** на платі знижує до 5 В. Часто під'єднують **батарейку** **9 В** з кліпсою зі штекером, щоб користуватися Arduino далеко від комп'ютера.
- **GND** (*ground*, "земля" або "маса"): це **негативний** **полюс** живлення, його також називають "спільним" або "0 вольт". На платі є кілька точок GND, до яких можна під'єднатися.
- **5V**: цей вивід дає напругу 5 В, **відрегульовану** **й** **стабілізовану** Arduino, щоб живити невеликі зовнішні схеми.
- **Виводи** **від** **0** **до** **13**: 14 **цифрових** виводів, які програмно налаштовують як **входи** або **виходи**. Arduino — цифровий пристрій: ці виводи створюють або читають лише **високий** (5 В) або **низький** (0 В) рівень. Виводи **0** **і** **1** позначені також **RX** і **TX**: вони з'єднані з **послідовним** **портом** мікросхеми (тим, яким користується й USB), тож краще залишати їх вільними.

- **A0–A5**: шість виводів, які можуть читати **аналогові рівні** (будь-яку напругу від 0 до 5 В) і перетворювати їх на число для програми.

1.1 Мозок: мікроконтролер ATmega328

Більшість плат Arduino використовує мікроконтролери сімейства **ATmega**, розроблені компанією Atmel (сьогодні частина **Microchip**). Arduino Uno використовує **ATmega328**. Характеристики, на які дивляться, вибираючи мікроконтролер:

Характеристика	Що це	ATmega328 (Arduino Uno)
Швидкість	максимальна робоча частота	16 МГц
RAM	“енергозалежна” пам’ять для обчислень і змінних	2 КБ
Flash	пам’ять, у якій зберігається програма	32 КБ
EEPROM	“постійна” пам’ять для параметрів	1 КБ
GPIO	виводи для цифрових входів/виходів	14
Аналогові виводи	входи з аналого-цифровим перетворювачем	6
PWM	виводи, що створюють регульований прямокутний сигнал	6 (позначені ~)
Послідовні порти	лінії для обміну даними	1
Живлення	допустимі напруги для мікросхеми	від 1,8 до 5,5 В

Примітка

EEPROM має обмежену кількість записів: її використовують для налаштувань, які змінюються рідко. **PWM** (*Pulse Width Modulation*, широтно-імпульсна модуляція) служить, наприклад, для регулювання швидкості двигуна чи яскравості світлодіода. ATmega328 існує також у корпусі **DIL** (dual in line), який можна поставити на макетну плату: з кількома іншими компонентами можна зібрати власну плату з мікроконтролером. Для готових виробів часто використовують **ATtiny**, “молодших братів”, які можуть коштувати менше одного євро.

2 Структура програми

Визначення

Програма Arduino (*sketch*) має дві **функції**, яких ніколи не може бракувати:

- **`setup()`**: виконується **лише один раз**, під час запуску або коли Arduino отримує живлення. Містить **початкові налаштування** (напрямок виводів, швидкість послідовного порту, …);
- **`loop()`**: виконується **по колу, безкінечно**, після `setup()`. Містить саму роботу.

```
void setup()
{
  // код, що виконується лише один раз
}

void loop()
{
  // код, що повторюється безкінечно
}
```

Примітка

Коли програмують на C **поза Arduino** (наприклад, на комп'ютері), єдина обов'язкова функція — **`main()`**: саме з неї починається програма. Arduino "ховає" `main` і сам викликає `setup` і `loop`. Що таке функція і як писати власні, пояснено в матеріалі A01.

Увага

Правила запису мовою C, які діють завжди:

- кожна команда закінчується **крапкою з комою** `;`;
- **фігурні дужки** `{ }` охоплюють блок команд (тіло функції, `if`, циклу);
- **коментарі** не виконуються: `//` коментує до кінця рядка, `/* ... */` коментує все, що між ними, навіть у кількох рядках;
- C розрізняє **великі і малі літери**: `digitalWrite` правильно, `digitalwrite` — помилка.

3 Біти, байти і двійкові числа

Визначення

Біт — це двійкова цифра: вона дорівнює **0** або **1**. **Байт** складається з **8 бітів**. За допомогою n бітів можна подати 2^n різних значень: з 8 бітами — $2^8 = 256$ значень.

Приклад

З двійкового в десяткове: кожну цифру множать на її **вагу**, яка є степенем 2 (справа: $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, $2^4 = 16$, ...). Наприклад:

$$10010_2 = 1 \cdot 16 + 0 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 = 18$$

З десяткового в двійкове: послідовно ділять на 2 і записують остачі, потім читають їх **від останньої до першої**. Для 18: $18 : 2 = 9$ остача 0; $9 : 2 = 4$ остача 1; $4 : 2 = 2$ остача 0; $2 : 2 = 1$ остача 0; $1 : 2 = 0$ остача 1. У зворотному порядку: 10010.

4 Змінні і типи даних

Визначення

Змінна — це **контейнер** з назвою, у якому програма тимчасово зберігає значення для обчислень. Як **шухляди** **шафи**, змінні мають **різні** **розміри** залежно від чисел, які мають уміщати: тому кожна змінна має **тип**.

Тип (Arduino Uno)	Байти	Значення
`char`	1	від -128 до 127
`byte` (або `unsigned char`)	1	від 0 до 255
`int`	2	від $-32\,768$ до $32\,767$
`unsigned int`	2	від 0 до $65\,535$
`long`	4	приблизно від $-2{,}1$ до $+2{,}1$ мільярда
`float` (і `double`)	4	числа з комою, приблизно 6–7 значущих цифр

- **`char`** містить також **символ**: символи відповідають числам за **кодом** **ASCII**. Дві змінні нижче мають **однакове** **значення**, бо велика `A` в ASCII — це 65:

```
char c1 = 'A';
char c2 = 65;
```

  Символ пишуть в **одинарних** **лапках** (`'A'`); текст — у **подвійних** **лапках** (`"ciao"`).

- **`unsigned`** (без знака) означає, що змінна не містить від'ємних чисел: з тим самим обсягом вона досягає вдвічі більших додатних значень.
- На Arduino Uno `float` і `double` — **одне** **й** **те** **саме**: 4 байти.

Увага

Розмір типу **залежить** **від** **машини**: на Arduino Uno `int` займає 2 байти, на комп'ютері зазвичай 4. І увага до діапазону `char`: він **зі** **знаком** ($-128 \ldots 127$); від 0 до 255 — це тип `byte`. На питання "який розмір `char`?" відповідь — **1** **байт**, тобто 8 бітів.

4.1 Оголосити і присвоїти

```
int temperatura;        // оголошення: створюю змінну
temperatura = 21;       // присвоєння: кладу в неї 21
int conteggio = 0;      // оголошення і присвоєння разом
```

Примітка

Назви змінних: літери, цифри й нижнє підкреслення, без пробілів, **не** **можуть** **починатися** з **цифри** і не можуть бути **зарезервованими** **словами** C (як-от `int`, `if`, `for`, `while`, `void`, `return`). Вибирайте назви, які кажуть, що в змінній: `temperatura` краще, ніж `t`.

4.2 Сталі з `#define`

Значенню, яке ніколи не змінюється, можна дати назву за допомогою **`#define`** (без = і без крапки з комою). За домовленістю назву пишуть **великими літерами**:

```
#define LED_ROSSO 3
#define BASE 7
```

Відтоді програма може використовувати `LED_ROSSO` замість 3. Усі подробиці — у матеріалі A01.

4.3 Масиви

Визначення

Масив (*array*, вектор) — це послідовність значень **одного типу** під однією назвою. До кожного елемента звертаються за **індексом** у квадратних дужках. Індекс **починається з 0**: у масиві з 5 елементів індекси від 0 до 4.

```
int valori[2];             // масив з двох елементів
valori[0] = 42;            // задає перший елемент
valori[1] = -42;           // задає другий елемент

int altri[] = { 42, -42 }; // той самий вміст, записаний одразу
int primo = altri[0];      // primo дорівнює 42
```

Увага

- Останній допустимий індекс — **розмір** — 1: у `int v[5]` немає `v[5]`. Компілятор **не повідомляє** про помилку: програма читає або записує пам'ять, яка їй не належить, з непередбачуваними наслідками.
- Масив, оголошений **усередині функції** і не ініціалізований, містить **випадкові значення**: завжди ініціалізуйте його (наприклад, `int v[10] = {0};` заповнює його нулями).

5 Оператори

Визначення

Оператори — це символи, які пов'язують значення між собою. Ми бачили чотири групи: **присвоєння**, **математичні**, **порівняння** і **логічні**.

5.1 Присвоєння: =

Символ **=** **копіює** значення праворуч у змінну ліворуч. Це не питання ("чи вони рівні?"), а **наказ**. Його читають "бере": `risultato = a + b` читається "risultato бере a плюс b".

```
char risultato;
char a;
char b;
a = 5;
b = 6;
risultato = a + b;   // risultato дорівнює 11
```

5.2 Математичні оператори

+	додавання
-	віднімання
*	множення
/	ділення
%	остача від ділення (modulo)

Визначення

Між **цілими** **числами** ділення **/** дає лише **цілу** **частку**: частина після коми відкидається. Оператор **%**, який **не** **має** **нічого** **спільного** з відсотками, дає **остачу** від ділення:

$$5 / 2 = 2 \qquad 5 \,\%\, 2 = 1$$

Приклад

```
int a = 10;
int b = 2;
int c = a + b;   // 12
int d = a - b;   // 8
int e = a / b;   // 5
int f = a * b;   // 20
int g = 7 % 3;   // 1 (7 поділити на 3 --- це 2 і остача 1)
```

Остача дуже корисна: `n % 2` дорівнює 0, якщо `n` **парне**, і 1, якщо **непарне**.

5.3 Оператори порівняння

Вони порівнюють два значення. Результат завжди **істина** або **хиба** (у C: 1 або 0).

Символ	Значення	Приклад
>	більше	`5 > 3` — істина
<	менше	`7 < 3` — хиба
>=	більше або дорівнює	`5 >= 5` — істина
<=	менше або дорівнює	`2 <= 1` — хиба
==	дорівнює	`5 == 5` — істина
!=	не дорівнює	`9 != 3` — істина

Результат порівняння можна також записати в змінну: після `char a = 5 > 3;` змінна `a` дорівнює **істині** (1); після `char r = (7 != 7);` змінна `r` дорівнює **хибі** (0).

Увага

Найпоширеніша помилка в C: плутати `=` (присвоєння, "бере") з `==` (порівняння, "подвійне дорівнює"). Запис `if (x = 5)` **не порівнює**: він **присвоює** 5 змінній x, і умова завжди істинна.

5.4 Логічні оператори

Вони **поєднують** **кілька** **умов**, за тією самою логікою, що й логічні елементи:

Символ	Назва	Істина, коли...
&&	AND (і)	**обидві** умови істинні
||	OR (або)	**хоча** **б** **одна** умова істинна
!	NOT (не)	умова **хибна**

Приклад

`if (temperatura > 18 && temperatura < 25)`: істина, лише якщо температура між 18 і 25.
`if (tasto == 'x' || tasto == 'X')`: істина, якщо натиснули малу x *або* велику X.

6 Приймати рішення: `if` і `if ... else`

Визначення

`if` виконує блок коду **лише** **тоді**, коли умова **істинна**; якщо вона хибна, блок **пропускається**.

```
if УМОВА()
{
  // виконується, лише якщо УМОВА істинна
}
```

З `else` кажуть також, що робити, коли умова **хибна**: програма завжди виконує **один** **із** **двох** блоків.

```
if УМОВА()
{
  // якщо істинна
}
else
{
  // якщо хибна
}
```

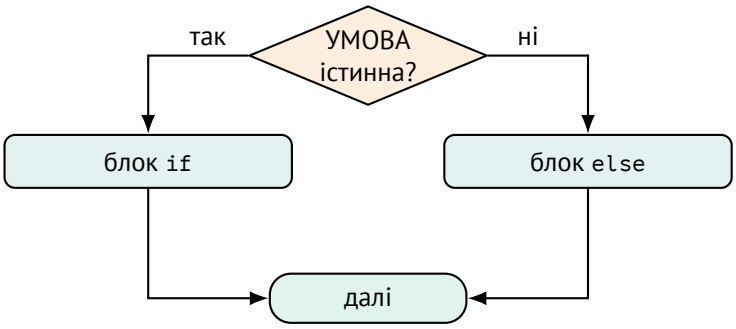


Рис. 2: Блок-схема `if ... else`: завжди проходять лише через один із двох блоків. Без `else` гілка “ні” йде одразу далі.

Приклад

```
int a = 9;
int b = 7;
if (a < b)
{
  Serial.println("a e' minore di b");
}
else
{
  Serial.println("a non e' minore di b");
}
```

Умова `a < b` (9 < 7) **хибна**, тому виконується блок `else`. З `a = 5` умова була б істинною, і надрукувалося б перше повідомлення.

Примітка

Якщо в блоці **лише одна команда**, фігурні дужки не обов’язкові, але варто **ставити їх завжди**: код зрозуміліший, і менше помилок. Можна писати кілька `if` один за одним: кожен перевіряється **окремо**, і істинними можуть виявитися навіть кілька.

7 Повторювати: `for`, `while`, `do ... while`

7.1 Цикл `for`

Визначення

`for` повторює блок коду **визначену кількість** разів:

```
for початок(; умова; приріст)
{
  // код, що повторюється
}
```

- **початок**: виконується **лише один раз**, на початку; зазвичай задають початкове значення змінної, яка рахує повторення;

- **умова**: перевіряється **перед** **кожним** **повторенням**; якщо вона істинна, блок виконується, якщо хибна — цикл закінчується;
- **приріст**: виконується **наприкінці** **кожного** **повторення**; змінює змінну, тож рано чи пізно умова стає хибною.

Приклад

```
for (int i = 0; i < 4; i++)
{
  // код повторюється 4 рази
}
```

`i++` означає `i = i + 1`. Змінна `i` набуває значень 0, 1, 2, 3: блок виконується **4** **рази**. Коли `i` стає 4, умова `i < 4` хибна, і цикл закінчується. Можна також рахувати назад (`i--`) або з іншим кроком (`i = i + 2`).

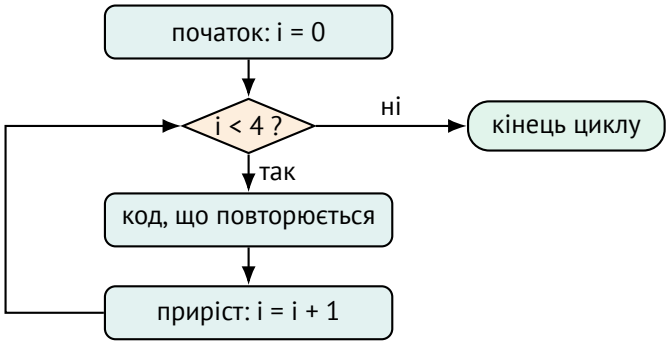


Рис. 3: Блок-схема циклу `for`.

7.2 Цикл `while`

Визначення

`while` (“поки”) повторює блок, **доки** умова залишається істинною. Умову перевіряють **перед** кожним повторенням: якщо вона хибна вже на початку, блок не виконується жодного разу.

```
int contatore = 1;            // початок
while (contatore <= 5)        // умова
{
  Serial.println(contatore);
  contatore = contatore + 1;  // приріст: без нього цикл не закінчиться
}
```

Програма друкує числа від 1 до 5.

Увага

Безкінечний **цикл**: якщо забути змінити змінну з умови, умова завжди залишається істинною, і цикл **ніколи** **не** **закінчується**.

Примітка

Є також **`do ... while`**, який перевіряє умову **наприкінці**: блок виконується **щонайменше один раз**. Увага до крапки з комою після умови: `do { ... } while (умова);`

Визначення

`for` чи `while`? Усе, що робиться одним, можна зробити іншим: вибір — питання **стилю**. Зазвичай **`for`** використовують, коли кількість повторень **відома наперед** (“увімкни світлодіод 10 разів”), а **`while`** — коли вона залежить від **зовнішніх подій** і з’ясовується лише *під час* виконання (“чекай, поки не натиснуть кнопку”).

8 Цифрові входи і виходи

Приклад

Двері супермаркету. У супермаркеті через одні двері заходять, через інші виходять. Виводи Arduino теж “двері”: через одні інформація **входить** (**input**), через інші **виходить** (**output**). На виході вирішує Arduino: увімкнено чи вимкнено. На вході вирішує зовнішній світ, а Arduino слухає. Як у нас **рот** (щоб говорити) і **вуха** (щоб слухати): вухами не говорять.

Визначення

- **Виконавчий пристрій** (*attuatore*) перетворює електричну величину на іншу фізичну величину (світлодіод — на світло, двигун — на рух): його під’єднують до виводу **output**.
- **Датчик** (*sensore*) перетворює фізичну величину на іншу, зазвичай на **напругу** або **струм**, які Arduino може виміряти (кнопка, датчик температури): його під’єднують до виводу **input**.

Інформація, що надходить, може бути **цифровою** (лише два стани: увімкнено/вимкнено, істина/хиба, HIGH/LOW) або **аналоговою** (будь-яке значення в певному діапазоні).

8.1 Команди

- **`pinMode(вивід, режим)`**: визначає, чи вивід є входом (`INPUT`) чи виходом (`OUTPUT`). Зазвичай пишуть у `setup()`. Приклади: `pinMode(3, OUTPUT);` для світлодіода, `pinMode(7, INPUT);` для кнопки.
- **`digitalWrite(вивід, стан)`**: ставить вихід на 5 В (`HIGH`, світлодіод **вмикається**) або на 0 В (`LOW`, світлодіод **вимикається**).
- **`digitalRead(вивід)`**: читає вхід і дає `HIGH` або `LOW`. Значення записують у змінну: `int stato = digitalRead(7);`
- **`delay(мс)`**: зупиняє програму на вказану кількість **мілісекунд** (`delay(1000)` = одна секунда). Приймає лише цілі числа; для коротшого часу є `delayMicroseconds(мкс)`, у мікросекундах.

9 Кнопка і резистори pull-up і pull-down

Визначення

Кнопка — найпростіший цифровий датчик. Вона поводиться двома способами: **розімкнене коло** (струм не проходить) або **коротке замикання** (струм проходить), залежно від того, натиснута вона чи ні. У продажу є безліч моделей, але символ на схемах завжди той самий.

Увага

Вхід, під'єднаний **лише** до кнопки, коли кнопка розімкнена, ні з чим не з'єднаний: кажуть, що він "**висить у повітрі**" (*flottante*), і читає випадкові значення (досить піднести руку). Тому вхід завжди треба "підтягнути" до певного рівня **резистором**: до 5 В (**pull-up**) або до GND (**pull-down**). Типове значення — **10 кОм** (10 кΩ).

9.1 Кнопка з резистором pull-up

З резистором **pull-up**, коли кнопка **не натиснута**, вхід читає **HIGH** (резистор з'єднує його з 5 В); коли кнопка **натиснута**, він читає **LOW** (кнопка з'єднує його з GND). Логіка **обернена**: натиснуто = LOW.

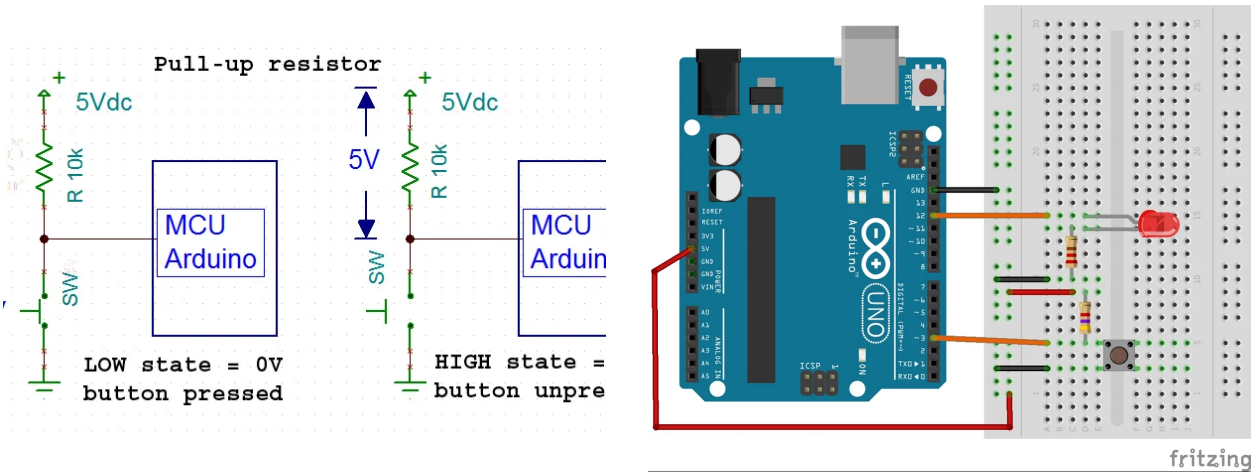


Рис. 4: Pull-up. Ліворуч схема: резистор 10 кΩ між виводом і 5 В, кнопка між виводом і GND; натиснуто = 0 В (LOW), не натиснуто = 5 В (HIGH). Праворуч монтаж на макетній платі, зі світлодіодом на виводі 12 і кнопкою на виводі 3.

```
/* перевірка резистора pull-up */
void setup()
{
  pinMode(3, INPUT);     // кнопка
  pinMode(12, OUTPUT);   // світлодіод
}

void loop()
{
  int statoPulsante = digitalRead(3);    // читає кнопку
  if (statoPulsante == LOW)              // pull-up: натиснуто = LOW
  {
    digitalWrite(12, HIGH);              // вмикає світлодіод
  }
  else
  {
    digitalWrite(12, LOW);               // вимикає світлодіод
  }
}
```

Примітка

Arduino має також **внутрішній резистор pull-up**, який вмикають так: `pinMode(3, INPUT_PULLUP);`. Тоді зовнішній резистор не потрібен, а логіка така сама, як у pull-up (натиснуто = LOW).

9.2 Кнопка з резистором pull-down

З резистором **pull-down**, коли кнопка **не натиснута**, вхід читає **LOW** (резистор з'єднує його з GND); коли кнопка **натиснута**, він читає **HIGH** (кнопка з'єднує його з 5 В). Логіка "природна": натиснуто = HIGH.

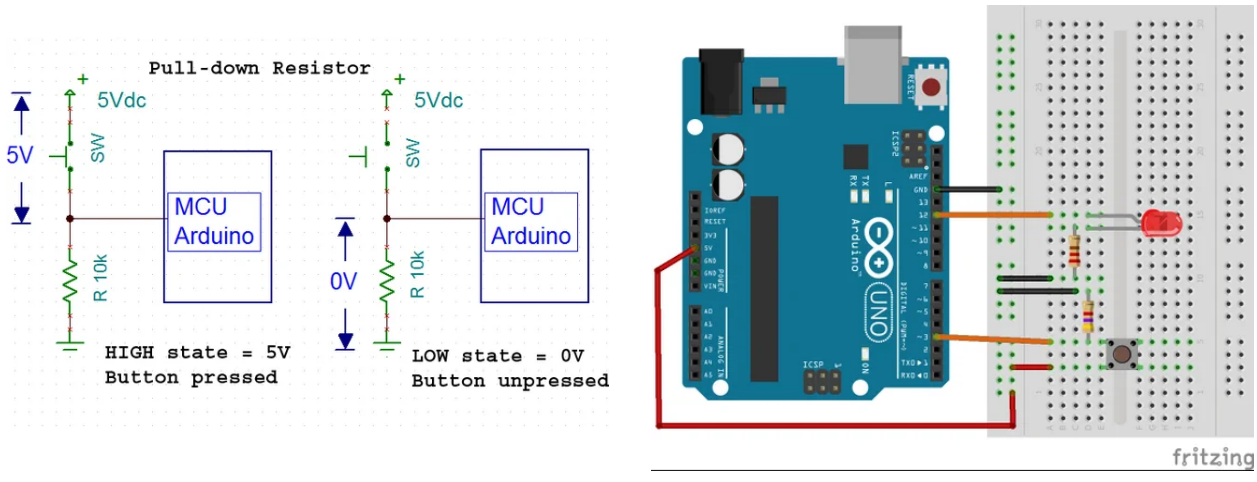


Рис. 5: Pull-down. Ліворуч схема: кнопка між 5 В і виводом, резистор 10 кΩ між виводом і GND; натиснуто = 5 В (HIGH), не натиснуто = 0 В (LOW). Праворуч монтаж на макетній платі.

```
/* перевірка резистора pull-down */
int pinPulsante = 3;
int pinLed = 12;

void setup()
{
  pinMode(pinPulsante, INPUT);
  pinMode(pinLed, OUTPUT);
}

void loop()
{
  int statoPulsante = digitalRead(pinPulsante);
  if (statoPulsante == HIGH)   // pull-down: натиснуто = HIGH
  {
    digitalWrite(pinLed, HIGH);
  }
  else
  {
    digitalWrite(pinLed, LOW);
  }
}
```

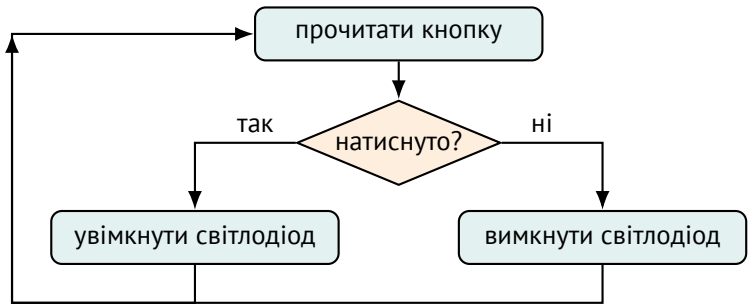


Рис. 6: Блок-схема двох програм із кнопкою. Відрізняється лише те, що означає "натиснуто": LOW з pull-up, HIGH з pull-down.

10 Послідовний зв'язок

Монітор послідовного порту (*monitor seriale*) — інструмент середовища Arduino, який показує текст, що надсилає Arduino, і дозволяє надсилати текст в Arduino. Його відкривають з меню *Strumenti* → *Monitor seriale* (Інструменти → Монітор порту) або іконкою з лупою вгорі праворуч.

10.1 Ініціалізація: `Serial.begin`

Визначення

Перш ніж користуватися послідовним портом, його треба **ініціалізувати**, лише один раз, тож у `setup()`:

```
Serial.begin(9600);
```

`Serial` — це **клас**, який керує зв'язком, **`begin`** — **метод**, який його налаштовує. Число — це **baud rate**, **швидкість** зв'язку: що воно більше, то швидше передаються дані. Можливі значення: 300, 600, 1200, 2400, 4800, **9600**, 14400, 19200, 28800, 31250, 38400, 57600, **115200**. Найуживаніші — 9600 і 115200.

Увага

У моніторі порту, унизу праворуч, є меню зі **швидкістю приймання**: вона має бути **такою самою**, як у `Serial.begin`, інакше читаються безглузді символи. Автоматично це не відбувається: налаштуйте самі.

10.2 Надсилання: `Serial.print` і `Serial.println`

```
void setup()
{
  Serial.begin(9600);
}

void loop()
{
  Serial.print("Ciao a tutti");
  delay(1000);   // щоб не забивати канал надто частими повідомленнями
}
```

`Serial.print` пише в монітор **без переходу на новий рядок**: повідомлення йдуть одне за одним в одному рядку. **`Serial.println`** пише, а потім **переходить на новий рядок**.

Увага

Якщо копіюєте код з документа (Google Документи, Word), **лапки** можуть бути "друкарськими" (“ ”), а для Arduino це **інший символ**, і буде помилка компіляції. У коді мають бути **прямі** лапки (): краще передрукувати код вручну.

10.3 Приймання: `Serial.available` і `Serial.read`

Приклад

Віконце і черга. Дані, що надходять з комп'ютера, потрапляють у пам'ять, яка називається **буфер**, як люди в **черзі** перед віконцем. **`Serial.available()`** каже, **скільки** даних у черзі (0, якщо нічого не надійшло). **`Serial.read()`** пропускає до віконця **першого** з черги: **читає** його і **прибирає** з черги.

Визначення

Спочатку перевіряють, **чи** щось надійшло, потім читають і вирішують, що з цим робити. Як і `digitalRead`, ці дві команди **дають** **результат**, який записують у змінну:

```
int x = Serial.available();   // скільки даних у черзі?
if (x != 0)                   // щось надійшло
{
  int b = Serial.read();      // читаю перше і прибираю з черги
  // ... вирішую, що робити, залежно від b
}
```

Приклад

Свій світлодіод для кожної літери. Залежно від отриманої літери (`c`, `f` або `u`) вмикається інший світлодіод, на виводах 2, 3 і 4:

```
void setup()
{
  Serial.begin(9600);
  pinMode(2, OUTPUT);   // світлодіод 1
  pinMode(3, OUTPUT);   // світлодіод 2
  pinMode(4, OUTPUT);   // світлодіод 3
}

void loop()
{
  int x = Serial.available();
  if (x != 0)
  {
    int b = Serial.read();
    if (b == 'c')
    {
      digitalWrite(2, HIGH);
    }
    if (b == 'f')
    {
      digitalWrite(3, HIGH);
    }
    if (b == 'u')
    {
      digitalWrite(4, HIGH);
    }
  }
}
```

Увімкнені світлодіоди залишаються увімкненими: щоб їх вимкнути, потрібна ще одна літера, з іще одним `if` і `digitalWrite(..., LOW)`. Інший приклад, зроблений у класі 15/10 ("ciao" і "premuto"), є в матеріалі A01.

11 Вправи для повторення

Приклад

1. Скільки байтів займає `char` на Arduino Uno? А `int`? Який діапазон значень у кожного?
2. Переведіть у десяткове двійкове число 1011, а в двійкове — десяткове число 13.
3. Чому дорівнюють `17 / 5` і `17 % 5` для цілих чисел?
4. Напишіть програму, яка блимає світлодіодом на виводі 8 10 разів за допомогою `for`, а потім зупиняється (підказка: `for` ставлять у `setup()`).
5. Під'єднайте кнопку з резистором pull-down до виводу 2: поки її натиснуто, світлодіод на виводі 13 має світити.
6. З послідовним портом: коли надходить літера `a`, світлодіод вмикається, коли `s` — вимикається. Щоразу Arduino відповідає в моніторі "acceso" (увімкнено) або "spento" (вимкнено).
7. Знайдіть помилку: `if (x = 3) { digitalWrite(5, HIGH); }`

12 Що запам'ятати

- Arduino Uno: USB (живлення і дані), гніздо для штекера, **GND**, **5V**, цифрові виводи **0–13** (0 і 1 для послідовного порту), аналогові входи **A0–A5**; мікроконтролер **ATmega328**.
- **`setup()`** один раз, **`loop()`** безкінечно; у "звичайному" C усе починається з `main()`. Крапка з комою, фігурні дужки, коментарі, великі й малі літери.
- 1 байт = 8 бітів = 256 значень. `char` 1 байт ($-128 \ldots 127$), `byte` 0…255, `int` 2 байти, `float` 4 байти, з комою. Масиви: індекси від 0 до розмір-1.
- Оператори: `=` присвоює; `+ - * / %` (`/` між цілими дає частку, `%` — остачу); порівняння `> < >= <= == !=`; логічні `&& || !`. Ніколи `=` замість `==`.
- `if` / `else`, щоб вирішувати; `for`, якщо повторення відомі, `while`, якщо вони залежать від зовнішніх подій.
- `pinMode`, `digitalWrite`, `digitalRead`, `delay`. Виконавчі пристрої на виходах, датчики на входах.
- Кнопка: завжди з **pull-up** (натиснуто = LOW) або **pull-down** (натиснуто = HIGH), 10 кОм; або `INPUT_PULLUP`.
- Послідовний порт: `Serial.begin` (така сама швидкість у моніторі), `print`/`println`, `available` (скільки в черзі), `read` (читає і прибирає з черги).

Глосарій: італійські слова, які вживають на уроці

Italiano	**Українською**
scheda, pin, alimentazione	плата, вивід, живлення
ingresso / uscita	вхід (input) / вихід (output)
variabile, tipo di dato	змінна, тип даних
assegnare	присвоювати
quoziente, resto	частка, остача
vero / falso	істина / хиба
ripetere, ciclo	повторювати, цикл
pulsante, premuto	кнопка, натиснуто
resistenza di pull-up / pull-down	резистор pull-up / pull-down
sensore, attuatore	датчик, виконавчий пристрій
monitor seriale, coda	монітор порту, черга
acceso / spento	увімкнено / вимкнено
verifica di ripasso	контрольна з повторення